

# RAG Pipeline Code Breakdown

## Overview

Your RAG system has two main scripts:

1. **rag\_db.py** — Takes a PDF → chunks it → embeds chunks → stores in ChromaDB
2. **rag\_query.py** — Loads the stored embeddings → searches them → streams LLM response

Data flows: PDF → Documents → Chunks → Vectors → ChromaDB (persisted) → Retrieval + Ranking → LLM → User

---

## Part 1: Database Creation (rag\_db.py)

### Step 1: Document Loading

```
python

@traceable(name="load_documents")
def load_documents(pdf_path: str) -> list:
    if os.path.isdir(pdf_path):
        loader = DirectoryLoader(pdf_path, glob="**/*.pdf", loader_cls=PyPDFLoader)
    else:
        loader = PyPDFLoader(pdf_path)
    documents = loader.load()
    return documents
```

### What it does:

- Takes a file path (single PDF) or folder path (multiple PDFs)
- PyPDFLoader extracts text from each page
- Returns a list of `Document` objects, each representing one page
- Each document has `.page_content` (the text) and `.metadata` (like `{"source": "file.pdf", "page": 0}`)

### Example output:

```
Document(
  page_content="Chapter 1: Introduction to ML...",
  metadata={"source": "MachineLearning.pdf", "page": 0}
)
```

**Key detail:** One document = one page. A 100-page PDF returns 100 documents. This is raw and unsegmented — no overlap, no cleanup yet.

---

## Step 2: Document Chunking

```
python

@traceable(name="split_documents")
def split_documents(documents: list) -> list:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    chunks = splitter.split_documents(documents)
    return chunks
```

### What it does:

- Takes the page-level documents and breaks them into smaller chunks
- Tries to split on smart boundaries in order: paragraph breaks → newlines → sentences → words → characters
- `chunk_size=1000` means each chunk is ~1000 characters max
- `chunk_overlap=200` means the last 200 chars of chunk N also appear at the start of chunk N+1 — prevents context from being cut off mid-thought

### Why overlap? If a sentence spans chunks without overlap:

```
Chunk 1: "The cat sat on..."
Chunk 2: "...the mat by the window."
```

The retriever might pull Chunk 1 alone, and you'd miss the ending. With overlap, you get context on both sides.

### Example:

```
A 100-page PDF (100 documents) might become 500-800 chunks.
Each chunk is ~1000 chars (roughly 200-250 words).
```

**Key detail:** The splitter also inherits metadata from the original documents, so each chunk still knows which page it came from:

```
Document(  
    page_content="This is a 1000-char excerpt...",  
    metadata={"source": "MachineLearning.pdf", "page": 5}  
)
```

### Step 3: Embedding & Storage

```
python  
  
@traceable(name="create_vector_store")  
def create_vector_store(chunks: list, persist_dir: str = "chromaDb") -> Chroma:  
    embedding_model = GoogleGenerativeAIEmbeddings(  
        model="text-embedding-004",  
        project=os.environ.get("GOOGLE_CLOUD_PROJECT"),  
        location="us-central1",  
        vertexai=True,  
    )  
  
    BATCH_SIZE = 200  
    vector_store = None  
    for i in range(0, len(chunks), BATCH_SIZE):  
        batch = chunks[i : i + BATCH_SIZE]  
        if vector_store is None:  
            vector_store = Chroma.from_documents(...)  
        else:  
            vector_store.add_documents(batch)  
  
    return vector_store
```

#### What it does:

1. Creates an embedding model instance (Vertex AI text-embedding-004)
  - This model converts text → 768-dimensional vector
  - Same embedding model you'll use in `rag_query.py`
2. Processes chunks in batches of 200 (Vertex AI's limit per API call)
3. For the first batch: creates a fresh ChromaDB instance and stores embeddings
4. For subsequent batches: adds more embeddings to the existing store

#### What's happening under the hood:

- Each chunk gets embedded: `"This is a chunk..."` → `[0.23, -0.15, ..., 0.08]` (768 numbers)

- Chroma stores three things:
  - The original text (`page_content`)
  - The vector (for similarity search)
  - The metadata (source, page number)
- All three are persisted to disk in the `chromaDb/` folder

### Why batching?

- API limits: Vertex AI caps requests at 250 documents per call
- Efficiency: batching = fewer API round-trips
- Cost: fewer API calls = lower bill

**Key detail:** `Chroma.from_documents()` initializes a new store. `add_documents()` appends to it. The check `if vector_store is None` ensures you only call `.from_documents()` once — calling it twice would create two separate stores.

---

### Step 4: Verification

```
python

@traceable(name="verify_store")
def verify_store(vector_store: Chroma) -> None:
    count = vector_store._collection.count()
    print(f"[Verify] ChromaDB contains {count} document(s).")
```

#### What it does:

- Quick sanity check: how many vectors are actually stored?
  - If you chunked 500 documents, you should see 500 vectors
  - If you see 0, something went wrong (bad API key, network issue, etc.)
- 

## Part 2: Query Interface (rag\_query.py)

### Step 1: Load the Embedding Model

```
python
```

```
embedding_model = GoogleGenerativeAIEmbeddings(  
    model="text-embedding-004",  
    project=os.environ.get("GOOGLE_CLOUD_PROJECT"),  
    location="us-central1",  
    vertexai=True,  
)
```

### What it does:

- Creates the same embedding model as in `rag_db.py`
  - **Critical:** Use the exact same model. If you embed your documents with model A and search with model B, similarity scores are nonsense.
  - This is stateless — it doesn't load anything from disk, just sets up the API client
- 

## Step 2: Load the Vector Store

```
python  
  
vector_store = Chroma(  
    persist_directory="chromaDb",  
    embedding_function=embedding_model,  
)
```

### What it does:

- Loads the `chromaDb/` folder you created in `rag_db.py`
- Initializes Chroma with the same embedding function
- Now the vector store knows: "When I get a query, use this embedding model to embed it"

### What's loaded:

- All 500 (or however many) chunks you stored
  - Their vectors
  - Their metadata
  - All from disk
- 

## Step 3: Initialize the LLM

```
python
```

```
llm = ChatGoogleGenerativeAI(  
    model="gemini-2.5-flash",  
    project=os.environ.get("GOOGLE_CLOUD_PROJECT"),  
    location="us-central1",  
    vertexai=True,  
    temperature=0.2,  
    max_output_tokens=1024,  
    streaming=True,  
)
```

### What it does:

- Creates a client for Gemini 2.5 Flash (the generative model)
- `temperature=0.2` → low randomness, more deterministic (good for factual answers)
- `max_output_tokens=1024` → cap the response length
- `streaming=True` → token-by-token output (for the "loading..." feeling in the terminal)

**Key detail:** This is separate from the embedding model. Embedding (text → vector) is for retrieval. The LLM (token generation) is for synthesis.

---

### Step 4a: Vector Retriever (Semantic Search)

```
python  
  
vector_retriever = vector_store.as_retriever(  
    search_type="mmr",  
    search_kwargs={  
        "k": 5,  
        "fetch_k": 20,  
        "lambda_mult": 0.5,  
    },  
)
```

### What it does:

1. When you query with text, it:
  - Embeds the query using the same embedding model
  - Finds the 20 closest vectors in ChromaDB (`fetch_k=20`)
  - Runs Maximal Marginal Relevance (MMR) to pick the 5 most relevant and diverse
2. Returns 5 chunks

### Why MMR?

**Key detail:** This is just semantic search. It doesn't care about exact keywords — it understands meaning.

---

## Step 4b: BM25 Retriever (Keyword Search)

```
python

stored = vector_store.get(include=["documents", "metadatas"])
doc_objs = [
    Document(page_content=text, metadata=meta or {})
    for text, meta in zip(stored["documents"], stored["metadatas"])
]
bm25_retriever = BM25Retriever.from_documents(doc_objs)
bm25_retriever.k = 5
```

### What it does:

1. Fetches all chunks from Chroma (as plain text + metadata)
2. Rebuilds them as Document objects (LangChain expects this format)
3. Builds an in-memory BM25 index (keyword-based)
4. Returns the top 5 keyword matches

### Why BM25?

- BM25 is fast and exact. If you search for "Python", it finds chunks with "Python" in them.
- Vector search can miss keywords (if you search for "ML" but the doc says "machine learning", embedding might not catch it).
- Hybrid (BM25 + vector) = best of both worlds.

**Key detail:** BM25 is built in-memory from the stored chunks — it doesn't persist. So every time you run `rag_query.py`, it rebuilds this index from scratch (takes ~1 second).

---

## Step 4c: Ensemble Retriever

```
python

ensemble_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.4, 0.6],
)
```

### What it does:

1. Runs both BM25 and vector retrieval in parallel
2. BM25 returns top 5, vector returns top 5 (could be different results)
3. Merges the lists, weighted 40% from BM25 and 60% from vector
4. Deduplicates (same chunk twice = keep it once, but raise its score)
5. Returns the combined top results

### Example:

```
BM25 retriever: [Chunk A, Chunk B, Chunk C, Chunk D, Chunk E]
Vector retriever: [Chunk C, Chunk F, Chunk G, Chunk H, Chunk I]

Merged + weighted:
  Chunk C (appears in both): high score
  Chunk A (BM25 only): medium score
  Chunk F (vector only): medium score
  ...
```

---

## Step 4d: Cross-Encoder Reranker

```
python

cross_encoder = HuggingFaceCrossEncoder(
    model_name="cross-encoder/ms-marco-MiniLM-L-6-v2"
)
reranker = CrossEncoderReranker(model=cross_encoder, top_n=3)

retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=ensemble_retriever,
)
```

### What it does:



1. Takes the ensemble retriever's results (maybe 8-10 merged chunks)
2. Loads a specialized model (`ms-marco-MiniLM-L-6-v2`) — tiny, fast, trained to rank (query, chunk) pairs
3. Scores each (query, chunk) pair and re-orders them
4. Keeps only the top 3

### Why reranking?

- Retrieval is fast but imprecise. BM25 + vector might return 8 results, only 3 are actually good.
- Cross-encoder is slow but accurate. It understands the relationship between query and chunk.
- You compromise: use fast retrieval to get candidates (100 chunks → 8), then slow reranking on the candidates (8 → 3).

### Cost/benefit:

- +1 LLM call (the cross-encoder model), but it's tiny (6M params) so it's fast and free (runs locally)
- In exchange, your final 3 chunks are much higher quality

**Key detail:** The model is downloaded once and cached locally. No API calls.

---

### Step 5: Prompt Template

```
python
```

```

prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a precise and helpful AI assistant.

Rules:
- Answer ONLY from the provided context.
- Be concise and structured in your response.
- If the answer is not in the context, say:
    "I could not find the answer in the provided documents."
- Do not hallucinate or add information not present in the context.
- Cite relevant details where useful.
"""),
    ("human", """"Context:
{context}

Question: {question}

Answer: """)
])

```

### What it does:

- Defines the message format sent to the LLM
- System message: instructions for the AI (stay in context, don't make stuff up, be concise)
- Human message: the actual prompt template with placeholders `{context}` and `{question}`

### How it's used later:

- `{context}` gets replaced with the retrieved chunks (formatted)
- `{question}` gets replaced with the user's query
- The whole thing is sent to the LLM

## Step 6: Format Retrieved Chunks

```

python

def format_docs(docs: list[Document]) -> str:
    parts = []
    for i, doc in enumerate(docs, 1):
        source = doc.metadata.get("source", "unknown")
        page = doc.metadata.get("page", "?")
        parts.append(f"[Chunk {i} | {source}, p.{page}]\n{doc.page_content}")
    return "\n\n---\n\n".join(parts)

```

### What it does:

- Takes the 3 retrieved chunks (Document objects)
- Formats each one with a header showing source file and page number
- Joins them with `---` separators
- Returns a formatted string ready to drop into the prompt

### Example output:

```
[Chunk 1 | MachineLearning.pdf, p.5]
Neural networks are inspired by biological neurons...

---

[Chunk 2 | MachineLearning.pdf, p.12]
The backpropagation algorithm updates weights...

---

[Chunk 3 | MachineLearning.pdf, p.18]
Gradient descent minimizes the loss function...
```

---

## Step 7: Build the RAG Chain

```
python

rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough(),
    }
    | prompt
    | llm
    | StrOutputParser()
)
```

**What it does:** This is a LangChain "runnable" chain — a pipeline of transformations. Let's trace the flow:

1. **Input:** A user query string (e.g., "What is backpropagation?")
2. **Runnable dict:** `{"context": ..., "question": ...}`
  - `"context": retriever | RunnableLambda(format_docs)`
    - Passes the input through the retriever → 3 chunks

- Pipes them through `format_docs()` → formatted string
  - `"question": RunnablePassthrough()`
    - Just passes the input through unchanged (the query string)
  - Result: `{"context": "formatted chunk text", "question": "What is backpropagation?"}`
3. **Prompt:** Substitutes `{context}` and `{question}` into the template
    - Result: A fully-formed message ready for the LLM
  4. **LLM:** Sends the message to Gemini, gets tokens back
    - Result: Streamed tokens (if `streaming=True`)
  5. **StrOutputParser():** Collects the streamed tokens into a single string
    - Result: Final answer
- 

## Step 8: Query Function

```
python
@traceable(name="rag_query", metadata={...})
def run_query(question: str) -> str:
    result = []
    print("AI Assistant:")
    for chunk in rag_chain.stream(question):
        for ch in chunk:
            print(ch, end="", flush=True)
        result.append(chunk)
    print()
    return "".join(result)
```

### What it does:

1. Decorates the function with `@traceable` — logs every call to LangSmith
2. Calls `rag_chain.stream(question)` — streams tokens one by one (instead of waiting for the whole response)
3. Prints tokens in real-time as they arrive (streaming UX)
4. Collects all tokens in `result` list
5. Returns the full answer as a string

### Why streaming?

- User sees output immediately ("loading..."), not a long wait then a wall of text

- Feels responsive, even on slow networks
- 

## Step 9: Query Loop

```
python

if __name__ == "__main__":
    print("RAG System Ready")
    while True:
        user_query = input("\nUser:\n").strip()
        if not user_query:
            continue
        if user_query == "0":
            break
        run_query(user_query)
```

### What it does:

- Infinite loop: prompt user for input
  - If input is "0", break and exit
  - Otherwise, pass to `run_query()` and display the streamed response
  - Loop again
- 

## Data Flow Summary

Here's how data moves through the whole system end-to-end:

### Indexing (rag\_db.py):

```
PDF file
  ↓ (PyPDFLoader)
Page documents (1 doc = 1 page)
  ↓ (RecursiveCharacterTextSplitter)
Chunks (1000 chars, 200 char overlap)
  ↓ (GoogleGenerativeAIEmbeddings)
Vectors (768 dimensions each)
  ↓ (ChromaDB)
Persistent vector store on disk (chromaDb/)
```

### Querying (rag\_query.py):

```
User query: "What is backpropagation?"
```

```
↓ (Ensemble Retriever)
├─ BM25 search → 5 keyword matches
└─ Vector search (MMR) → 5 semantic matches
↓ (Deduplicate & merge)
~8 candidate chunks
↓ (Cross-Encoder Reranker)
3 top-ranked chunks
↓ (format_docs)
"[Chunk 1 | ...]\nBackprop is...\n---\n[Chunk 2 | ...]..."
↓ (Prompt template substitution)
Full prompt: "You are... Context: [chunks] Question: What is backprop? Answer:"
↓ (Gemini LLM)
Streamed tokens: "Backprop is the algorithm..." (token by token)
↓ (collect & print)
Final answer
```

---

## Key Insights

1. **Same embedding model everywhere** — documents indexed with model A, query searched with model A. Critical for correctness.
  2. **Metadata travels with chunks** — source and page number stay attached, so you can cite where the answer came from.
  3. **Hybrid retrieval** — BM25 (keywords) + vector (meaning) together are better than either alone.
  4. **Reranking is the cheap boost** — uses a tiny local model to dramatically improve top-3 precision without much latency.
  5. **Streaming makes it feel fast** — even if the LLM takes 5 seconds, tokens appearing one-by-one feel responsive.
  6. **LangSmith tracing is silent but always on** — every function call is logged; you can analyze latency, debug issues, and replay traces.
- 

## Common Questions

**Q: Why do I need overlap in chunking?** A: Context. If a sentence spans two chunks and you split at the sentence boundary, the second chunk starts mid-thought. Overlap ensures both chunks have context on both sides.

**Q: What if I change the embedding model?** A: You must re-index. The vectors were computed with the old model and won't be comparable with the new one. This is a full rebuild: `rag_db.py` again from scratch.

**Q: Can I use different retriever weights?** A: Yes. Try `weights=[0.3, 0.7]` to favor vector search more, or `[0.5, 0.5]` for equal weight. Experiment with your data.

**Q: Why keep only 3 chunks after reranking?** A: Token budget. The prompt has to fit in the LLM's context window. 3 chunks (~3000 chars) + query + system message  $\approx$  ~700 tokens. Leaves room for a long answer. If you need more context, increase `top_n=5` or `top_n=7`, but watch for context window overflow.

**Q: Can I run this without Vertex AI?** A: Yes, swap the embedding model and LLM. Use OpenAI, Anthropic, or Ollama for local. The structure stays the same — only the API client changes.